tidySTL: Exposing Implicit Semantics in STL Evaluation

Sota Sato[1][0000-0001-7147-3989]✉ and Masaki Waga[2][0000-0001-9360-7490]

[1] AIST, Tokyo, Japan
sota.sato@aist.go.jp
[2] Kyoto University, Kyoto, Japan

Abstract. Robust semantics of Signal Temporal Logic (STL) provides a verdict, optimization objective, and training feedback across monitoring, falsification, and control. Evaluating a discretely sampled trace, however, requires decisions that neither the STL formula nor the samples fully specify, such as interpolation and end-of-trace behavior. We call these evaluator-level choices *implicit semantics*; they can vary across tools, altering robustness values and downstream results. We present tidySTL, a diagnostic toolkit that lets users evaluate the same formula and sampled trace under named behaviors of existing STL tools. By aligning intermediate robustness values along the shared formula structure, tidySTL localizes where two tool behaviors first diverge. We demonstrate this localization with compatibility backends for existing STL tools. The same architecture also serves as an extensible evaluator layer: new evaluation algorithms or semantics require only modular backend changes, while keeping the formula, input trace, and public interface fixed.

1 Introduction

Signal Temporal Logic (STL) is a standard formalism for specifying temporal properties of real-valued signals [9,7]. Its quantitative *robust* semantics measures how strongly a behavior satisfies or violates a specification, and the resulting margin underpins runtime monitoring [2], falsification [1], and learning-enabled design [12].

The robust semantics is defined over an idealized continuous signal, yet tools usually evaluate it on finite, discretely sampled traces. This requires evaluator-level choices that are not determined by the STL formula and samples alone, such as interpolation and end-of-trace behavior. We call these choices *implicit semantics*; Table 1 organizes the main sources of divergence.

Because these choices are often embedded inside each tool, two STL tools can report different robustness values or Boolean satisfaction verdicts for the same formula and trace. Such discrepancies have appeared in practice: validation in the ARCH-COMP 2021 falsification-tool competition had to account for tool-dependent robustness interpretations and interpolation conventions, with mismatches producing incorrect preliminary results and numerous validation issues [6]. These cases show that cross-tool discrepancies are not only a matter of agreement, but also of diagnosis.

To make such discrepancies actionable, we compare evaluator behaviors not only at the final output but also along the formula structure. This turns cross-tool discrepancies into diagnostic targets: the question is not only whether two tools agree, but where their evaluations first diverge.

We propose tidySTL,[3] a diagnostic toolkit for localizing such discrepancies. It evaluates the same formula and trace under named tool behaviors while aligning intermediate robustness values along a shared formula structure (Figure 1). This allows tidySTL to localize where two evaluator behaviors first diverge.

This paper makes the following contributions.

- We organize the implicit semantic choices in STL robustness evaluation and show that they can affect robustness values and Boolean satisfaction verdicts (Section 2 and Table 2).
- We propose tidySTL, with a common interface and representation for STL evaluator behavior, allowing both existing tools and new evaluator variants to share a single formula-and-trace frontend (Section 3 and Figure 1).
- We turn cross-tool discrepancies into diagnosable evaluation differences by localizing where two evaluator behaviors first diverge. We demonstrate this with compatibility backends for Breach [4], TaLiRo [1], and RTAMT [11] (Section 4.2 and Table 5).

Related Work. Quantitative STL robustness is used in monitoring, falsification, control, and learning and implemented in many tools. Most follow the classical robustness of Donzé and Maler [5] or Fainekos and Pappas [7]. Classical implementations include Breach for verification and parameter synthesis [4], S-TaLiRo for falsification [1], and RTAMT for online monitoring [11]; tidySTL compares evaluator-level choices across these uses. Beyond classical robustness, some tools further implement extended semantics or logics: STLCG++ [8] for smooth robustness, mstlo [14] for the RoSI [3]-style finite-prefix uncertainty, CauMon [15] for causation, and MoonLight [10] for spatio-temporal properties. Supporting a broader range of these semantics is future work.

2 Implicit Semantic Choices in STL Evaluation

We take the quantitative STL semantics of Donzé and Maler [5] as the representative theoretical reference. Together with the closely related framework by Fainekos and Pappas [7], it represents the practical setting used by most of the existing STL monitoring and falsification tools [6]; our focus is on evaluator-level variations within this setting.

To fix the scope, we recall only the definitions needed to delimit where implicit semantic choices arise in evaluation; the full definitions can be found in the reference papers [5,7].

Definition 1 (Signal). *A* signal *over a set of variables* $\mathbf{Var}$ *is a function* $\sigma : \mathbb{R}_{\geq 0} \to \mathbb{R}^{\mathbf{Var}}$.

[3] Artifact repository: `https://github.com/midoriao/tidystl`.

Table 1. Representative *implicit choices* in implementing STL robustness evaluation. These choices can affect both reported robustness values and the accepted class of formula-trace inputs. Further formal details are given in the extended version [13].

Axis	Choice
Finite-trace interpretation	Reading samples as a signal, affects robustness
Signal model	Piecewise-linear | Piecewise-constant | Discrete
Signal beyond horizon	Clamp | Pessimistic
Robustness at horizon	Exact | Copy-prev
Predicate semantics	Atomic-level semantics and supported forms
Distance metric	Signed margin | Euclidean (affects robustness)
Nonstandard predicates	E.g., Equality (affects acceptance/robustness)
Logic fragment	Which parts of the full logic are supported, affects acceptance
Unbounded intervals	Unbounded operators like $\square_{[0,\infty)}$ supported?
Until/release	$\mathcal{U}_I$ and $\mathcal{R}_I$ supported?
Open intervals	Bounds such as (a, b) and $(a, b]$ supported?
Top/bottom	$\top(+\infty)$ and $\bot(-\infty)$ supported?
Real-valued bounds	Non-integer temporal bounds supported?
Implementation-specific	Conventions affecting robustness or acceptance
Off-grid windows	Exact endpoints | Samples only | Empty-window fallback
Non-uniform time	As-is | Force equal spacing | Reject
Infinity encoding	IEEE $\pm\infty$ | Finite sentinels

Definition 2 (STL syntax). *STL formulas are generated by the following grammar:*

$$\varphi ::= \top \mid f(\boldsymbol{x}) \geq 0 \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 \wedge \varphi_2 \mid \varphi_1 \,\mathcal{U}_I\, \varphi_2 \mid \varphi_1 \,\mathcal{R}_I\, \varphi_2 \mid \Diamond_I \varphi \mid \square_I \varphi.$$

Here f is a function symbol, for example $f(x, y) = 4x - y - 3$, and I is a non-singular interval in $\mathbb{R}_{\geq 0}$, for example $[2, 5]$, $(1, 3]$, or $[0, \infty)$.

Informally, the robust semantics $\rho(\varphi, \sigma, t)$ replaces the Boolean verdict with a signed real-value, e.g., $\rho(\mathtt{x} - 3 \geq 0, \sigma, t) = \sigma_{\mathtt{x}}(t) - 3$. The Boolean connectives and temporal operators then aggregate the robustness values using min, max, inf, and sup; for example, $\rho(\square_{[0,2]}\,(\mathtt{x} - 3 \geq 0), \sigma, t) = \inf_{t' \in [t,\, t+2]}(\sigma_{\mathtt{x}}(t') - 3)$.

We call a design choice in a concrete evaluator's implementation an *implicit semantic choice* if the choice can change the resulting robustness evaluation even for the same formula and trace. For example, an evaluator often receives only a finite *timed state sequence* $(t_0, \boldsymbol{v}_0), \ldots, (t_n, \boldsymbol{v}_n)$ of timestamped samples, whereas the reference semantics is defined over an idealized signal. How this finite representation is interpolated into a signal over $\mathbb{R}_{\geq 0}$ is an implicit semantic choice. Table 1 organizes the choices we consider.

Example 1 (Verdict flip from one implicit choice). Consider the *signal beyond horizon* choice of Table 1. Let $\varphi = \Diamond_{[2,4]}(\mathtt{x} \geq 0)$ be evaluated at $t = 0$ on a

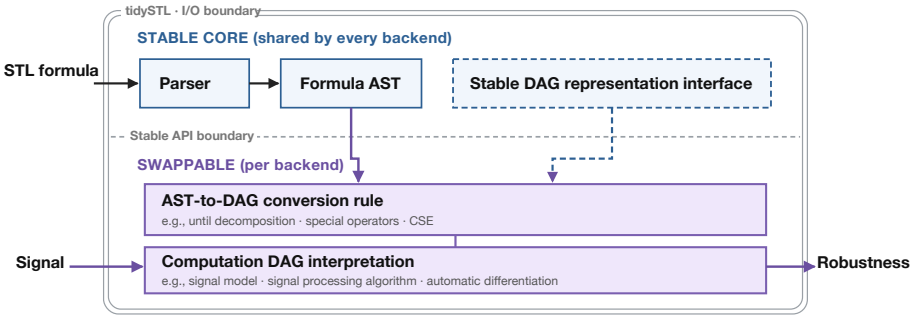


Fig. 1. Architecture of tidySTL: a stable core shared by every backend and a swappable backend layer (lowering and node interpretation) that isolates the implicit semantic choices.

trace ending at $T = 1$, whose final sample assigns 3 to `x`. The temporal window $[2, 4]$ lies entirely beyond the trace. A clamping rule treats the last sample as persisting and reports $+3$ (satisfied), while a pessimistic rule treats the window as unavailable and reports $-\infty$ (violated). Thus, the same formula and trace can receive an opposite Boolean verdict depending on this choice.

Crucially, neither rule is intrinsically wrong in Example 1: each is a defensible reading of a finite trace, which under-determines the idealized signal scored by the reference semantics. Thus, a cross-tool discrepancy need not be settled by declaring one value correct; it can instead be diagnosed by localizing the implicit choice responsible for it.

3 Design and Implementation

This section turns the implicit semantic choices of Section 2 into inspectable implementation structure. tidySTL exposes robustness evaluation through a uniform interface, while placing choices that vary across evaluators behind a backend boundary.

3.1 Evaluation Interface

tidySTL has two explicit inputs: formula and signal. A *formula* is a typed syntax tree, parsed from the textual STL frontend. A *signal* contains one or more named, batched value sequences over a shared time grid.

An optional third input, the *backend*, encapsulates the implicit semantic choices and the execution strategy. The call `robustness(formula, signal, backend)` returns an array of robustness values over time:

```python
from tidystl import Signal, parse, robustness
import numpy as np

phi = parse("G[0,5](F[0,2](velocity >= 10))")

times = np.linspace(0, 10, 200)
velocity_batch = np.array([np.sin(times), np.cos(times)])  # two traces
sig = Signal.from_dict(
    times=times,
    values={"velocity": velocity_batch},
)
rho = robustness(phi, sig)                    # default backend
rho = robustness(phi, sig, backend="breach")  # Breach-compatible
```

Keeping this interface small serves two purposes. For users, the same formula and signal can be evaluated under different documented backends without changing the surrounding application. For backend authors, new evaluation behavior can be implemented behind a stable interface rather than requiring a new parser, signal representation, or calling convention.

3.2 Architecture

Internally, tidySTL separates a stable core from backend-specific evaluation logic, as shown in Figure 1. The stable core is shared by every backend, and it does not decide how (φ, σ) maps to a robustness value. Those decisions are delegated to the backend layer.

This separation provides *observability*: because two backends build their intermediate outputs on the same shared formula (syntax-tree) representation, those outputs can be aligned at corresponding formula nodes. A post-order walk can therefore identify minimal divergent nodes, whose primitive operations and intermediate signals remain available for inspection. Section 4.2 demonstrates this diagnostic workflow.

Each backend lowers the formula AST into a backend-specific computation DAG (directed acyclic graph), and then interprets its typed primitive operations as a concrete signal-processing pipeline. This keeps signal-processing factors separate from the syntactic and recursive handling of STL formulas. Figure 2 illustrates such a lowering on an example formula with nested temporal operators.

Extension points. A backend defines how the formula is lowered into a computation DAG and how the resulting DAG nodes are interpreted. Extensions may introduce new node types during lowering or override how existing nodes are interpreted. The artifact repository includes the user manual (`docs/usage.md`) and reference implementations. In Section 4.1 we show that emulating an existing tool requires only compact backend-specific changes.

Architectural byproducts. Beyond this paper's scope of cross-tool discrepancy diagnosis, the swappable architecture (Figure 1) also enables new evaluation

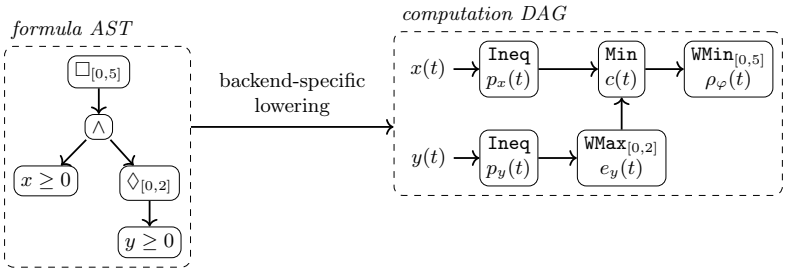


Fig. 2. Lowering of $\varphi :\equiv \square_{[0,5]}\big((\mathtt{x} \geq 0) \wedge \Diamond_{[0,2]}(\mathtt{y} \geq 0)\big)$ from AST (left) to computation DAG (right). Each DAG node is a primitive operation: the conjunction reduces pointwise while the temporal operators aggregate over their annotated window. Edges carry intermediate robustness signals that remain inspectable in the evaluation result.

strategies to be implemented through the same interface: the codebase ships differentiable (smooth robustness [8]) and SIMD-optimized backends as instances of this extensibility.

4 Empirical Study

We evaluate tidySTL 0.2.0 in two steps.[4] First, we show that realizing an implicit choice is a modular change confined to the backend layer, and that the resulting compatibility backends faithfully reproduce existing tools; we also assess runtime performance (Section 4.1). Second, building on these validated backends, we localize cross-tool discrepancies to the responsible choice and formula node (Section 4.2).

We fix a single set of reference tools throughout: Breach 1.11.4, RTAMT 0.3.5 (discrete default and dense, RTAMT_{den}), and TaLiRo (SVN revision 146). The implicit choices of Section 2 already produce discrepancies on them: Table 2 profiles three finite-trace axes using diagnostic cases that isolate one choice at a time, and each axis splits the tools into at least two observable behaviors.

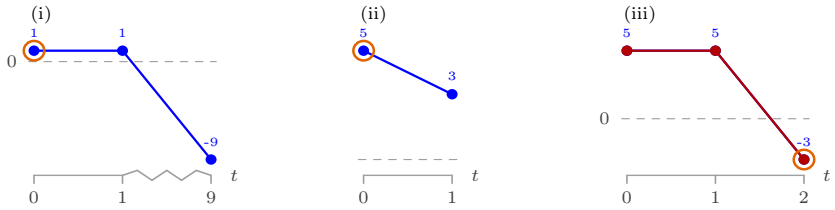
4.1 Backend Modularity, Compatibility, and Performance

We first measure the implementation cost of realizing an implicit choice in tidySTL. Table 3 shows that the shipped compatibility backends modify only formula-to-DAG lowering and DAG-node interpretation, leaving the stable core unchanged.

Compatibility validation. Each reproduces its tool's robustness on the operator-coverage regression suite—per-timestep for the Breach- and RTAMT-compatible

[4] The artifact repository includes the implementation used in our empirical study, together with scripts and documentation (`extra/experiments/`).

Table 2. Implicit-choice profile across three finite-trace axes (cf. Table 1) for Breach, TaLiRo, and RTAMT (discrete and dense modes). *Top*: the three diagnostic input signals, named next to each row's test formula; samples are spaced evenly by index (the t-axis gives each sample's real time); the orange ring marks the robustness readout time. *Bottom*: each row varies only the named choice; each cell gives the implicit choice revealed at the readout sample, with the observed value in parentheses. *n/o*: choice not observable in TaLiRo's whole-trace scalar output.



Implicit choice (test formula & signal)	Breach	TaLiRo	RTAMT	RTAMT$_{den}$
Signal model	dense	dense	discrete	dense
`G[0,2](x>=0)` (i)	(1)	(1)	(−9)	(1)
Signal beyond horizon	clamp	pessimistic	pessimistic	clamp
`F[2,4](x>=0)` (ii)	(3)	(−∞)	(−∞)	(3)
Robustness at horizon	copy-prev	*n/o*	exact	exact
`(x>=0) and (y>=0)` (iii)	(5)		(−3)	(−3)

Table 3. Implemented compatibility backends: layer changed and variant-specific LOC (blank/comment lines excluded) as an auditable size proxy.

Variant	Layer changed	LOC
Breach-compatible	node-interpretation overrides	85
TaLiRo-compatible	own lowering + node interpretation	167
RTAMT-compatible	own lowering + node interpretation	191

backends, and at $t = 0$ only for TaLiRo, which reports just that value—validated case by case against the tool's committed reference fixtures by the artifact's reproduction test suite.

Performance. We time $\square_{[0,5]}((\mathtt{x} > 0) \wedge \Diamond_{[0,2]}(\mathtt{y} > 0))$ on synthetic signals sampled at equally spaced time points over $[0, 10]$; Table 4 reports single and batched results at trace lengths $T = 101, 1001$. On this benchmark, per-node inspectability does not impose prohibitive overhead; the inspectable backends are competitive with RTAMT and STLCG++ on single traces. Our compatibility backends also scale well to batches. Though detailed performance analysis is beyond the scope of this paper, the results suggest that vectorization across traces is effective. Isolating signal processing from AST traversal makes such a design easier to realize.

Table 4. Mean evaluation time in ms (10 runs). Entries are $N = 1$ ($N = 256$); "-c." denotes a tidySTL compatibility backend. RTAMT's parenthesized values use a Python loop, whereas STLCG++ uses its native `torch.vmap` support.

T	Breach-c.	TaLiRo-c.	RTAMT-c.	RTAMT	STLCG++
101	0.6 (1.1)	0.5 (1.0)	0.3 (0.9)	0.2 (52.0)	0.2 (6.1)
1001	5.4 (26.9)	4.6 (25.2)	2.3 (23.9)	1.8 (510.2)	3.7 (545.7)

Table 5. Tool-vs-tool localization without ground truth: tool pair, size/depth, root robustness (ρ, left/right), and minimal divergent operator. Formulas are given in Examples 2 and 3; `window_sampling` is visualized in Figure 3.

Case	Tools compared	$\lvert\varphi\rvert$/depth	ρ	Divergence
`until_boundary`	Breach vs. RTAMT	3 / 1	-5 / $+2$	verdict flip on $\mathcal{U}_{[1,2]}$
`window_sampling`	Breach vs. TaLiRo	10 / 3	$+2$ / $+5$	value gap on $\square_{[1,2]}$

4.2 Localizing Cross-Tool Discrepancies

tidySTL localizes each discrepancy in Table 2 to the point where the two evaluations first diverge. Using the validated backends of Section 4.1, the localizer compares a diverging pair node by node: the backends expose inspectable outputs aligned to one shared syntax tree, so a single post-order walk reports a *minimal divergent node*—a lowest node whose backend outputs disagree while all of its children still agree—and the backend primitive used to evaluate that node (Table 5). Backend pairs whose lowerings differ structurally are compared at the shared syntax-tree level; predicate-internal arithmetic is treated as a leaf.

Example 2 (Off-grid window localization). Consider the following CPS-style requirement:

$$\square_{[0,3]}\Big((\texttt{mode} \geq 0) \wedge \Diamond_{[0,1]}\big((\texttt{enable} \geq 0) \wedge \square_{[1,2]}(\texttt{signal} \geq 0)\big)\Big) \wedge (\texttt{safety\_margin} \geq 0).$$

It can be read as a small CPS-style monitoring requirement with a mode signal, an enabling signal, a monitored continuous signal, and a safety margin. At $t = 0$, it requires a nonnegative safety margin and the required mode throughout $[0, 3]$; from each time in that window, an enabled point must occur within the next unit, after which the monitored signal must remain above its threshold over offsets $[1, 2]$. The localizer thus exposes an off-grid window choice three temporal levels below the root; as detailed in Figure 3, the discrepancy would be invisible from the Boolean verdicts alone.

Example 3 (Until verdict flip). A verdict can also flip outright. For the bounded-until formula $(\texttt{x} \geq 0)\,\mathcal{U}_{[1,2]}\,(\texttt{y} \geq 0)$ in `until_boundary`, Breach and RTAMT disagree on the root sign (-5.0 vs. $+2.0$); RTAMT is the lone outlier, with Breach and TaLiRo both reporting -5.0. The localizer pins every pairwise disagreement to the single $\mathcal{U}_{[1,2]}$ node—RTAMT's discrete-time until composition.

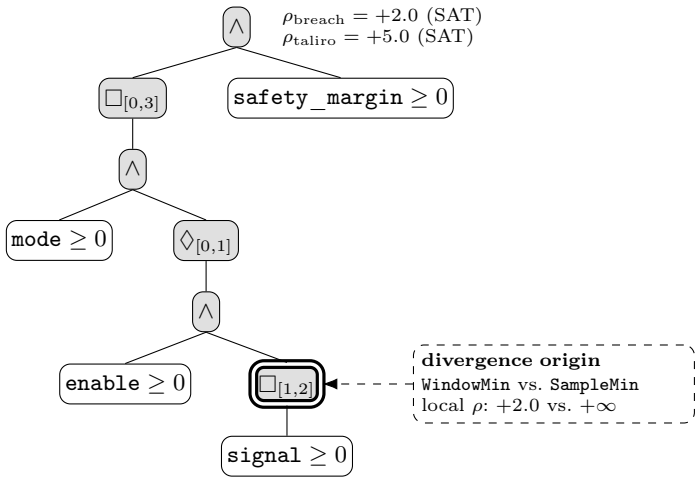


Fig. 3. Localization of the Breach–TaLiRo divergence in Example 2. The trace uses $t = 0, 3, 4, \ldots, 9$, `mode` = `enable` = 10, `safety_margin` = 9, and `signal` = $[3, 0, 5, \ldots, 5]$. At $t = 0$, the highlighted $\square_{[1,2]}$ window lies inside the $0 \to 3$ sampling gap: Breach interpolates `signal`$(1) = 2$, whereas TaLiRo finds no sample and returns $+\infty$. Shading traces this first disagreement from the double-outlined origin to the roots: Breach's $+2.0$ remains binding, whereas TaLiRo's vacuous branch leaves the next active constraint, $+5.0$, to determine the root. Both tools therefore satisfy the formula despite reporting different robustness.

This diagnosis mirrors the evaluator-level investigation required in ARCH-COMP to reconcile tool-dependent robustness and interpolation conventions [6].

Together, these examples show how tidySTL turns a numerical discrepancy into a semantic choice that users can assess. The appropriate choice is use-dependent: interpolation may match the physical signal model, conservative horizon handling may suit incomplete traces. When robustness values are compared or ranked, all runs should use one declared profile.

5 Conclusion and Future Work

tidySTL makes the implicit choices of STL robustness evaluation explicit at the evaluator boundary: by separating formula syntax, signal data, backend lowering, and DAG interpretation, it exposes choices otherwise buried in tool internals and mechanically localizes cross-tool discrepancies to minimal divergent formula nodes and responsible operations. This recasts robustness evaluation as an extensible semantic layer rather than an implementation detail of a particular monitor, falsifier, or learning pipeline; future work will broaden the supported semantics, including online evaluation, and expand the validation suite.

References

1. Annpureddy, Y., Liu, C., Fainekos, G.E., Sankaranarayanan, S.: S-TaLiRo: A tool for temporal logic falsification for hybrid systems. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS). Lecture Notes in Computer Science, vol. 6605, pp. 254–257. Springer (2011). https://doi.org/10.1007/978-3-642-19835-9_21
2. Bartocci, E., Deshmukh, J., Donzé, A., Fainekos, G., Maler, O., Ničković, D., Sankaranarayanan, S.: Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications. In: Lectures on Runtime Verification: Introductory and Advanced Topics, Lecture Notes in Computer Science, vol. 10457, pp. 135–175. Springer (2018). https://doi.org/10.1007/978-3-319-75632-5_5
3. Deshmukh, J.V., Donzé, A., Ghosh, S., Jin, X., Juniwal, G., Seshia, S.A.: Robust online monitoring of signal temporal logic. Formal Methods in System Design **51**(1), 5–30 (2017). https://doi.org/10.1007/s10703-017-0286-7
4. Donzé, A.: Breach, a toolbox for verification and parameter synthesis of hybrid systems. In: Computer Aided Verification (CAV). Lecture Notes in Computer Science, vol. 6174, pp. 167–170. Springer (2010). https://doi.org/10.1007/978-3-642-14295-6_17
5. Donzé, A., Maler, O.: Robust satisfaction of temporal logic over real-valued signals. In: Formal Modeling and Analysis of Timed Systems (FORMATS). Lecture Notes in Computer Science, vol. 6246, pp. 92–106. Springer (2010). https://doi.org/10.1007/978-3-642-15297-9_9
6. Ernst, G., Arcaini, P., Bennani, I., Chandratre, A., Donzé, A., Fainekos, G., Frehse, G., Gaaloul, K., Inoue, J., Khandait, T., Mathesen, L., Menghi, C., Pedrielli, G., Pouzet, M., Waga, M., Yaghoubi, S., Yamagata, Y., Zhang, Z.: ARCH-COMP 2021 category report: Falsification with validation of results. In: 8th International Workshop on Applied Verification of Continuous and Hybrid Systems (ARCH21). EPiC Series in Computing, vol. 80, pp. 133–152. EasyChair (2021). https://doi.org/10.29007/xwl1
7. Fainekos, G.E., Pappas, G.J.: Robustness of temporal logic specifications for continuous-time signals. Theoretical Computer Science **410**(42), 4262–4291 (2009). https://doi.org/10.1016/j.tcs.2009.06.021
8. Kapoor, P., Mizuta, K., Kang, E., Leung, K.: STLCG++: A masking approach for differentiable signal temporal logic specification. IEEE Robotics and Automation Letters **10**(9), 9240–9247 (2025). https://doi.org/10.1109/LRA.2025.3588389
9. Maler, O., Nickovic, D.: Monitoring temporal properties of continuous signals. In: Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT). Lecture Notes in Computer Science, vol. 3253, pp. 152–166. Springer (2004). https://doi.org/10.1007/978-3-540-30206-3_12
10. Nenzi, L., Bartocci, E., Bortolussi, L., Silvetti, S., Loreti, M.: MoonLight: A lightweight tool for monitoring spatio-temporal properties. International Journal on Software Tools for Technology Transfer **25**(4), 503–517 (2023). https://doi.org/10.1007/s10009-023-00710-5
11. Nickovic, D., Yamaguchi, T.: RTAMT: Online robustness monitors from STL. In: Automated Technology for Verification and Analysis (ATVA). Lecture Notes in Computer Science, vol. 12302, pp. 564–571. Springer (2020). https://doi.org/10.1007/978-3-030-59152-6_34
12. Raman, V., Donzé, A., Maasoumy, M., Murray, R.M., Sangiovanni-Vincentelli, A., Seshia, S.A.: Model predictive control with signal temporal logic specifications. In:

53rd IEEE Conference on Decision and Control (CDC). pp. 81–87. IEEE (2014). https://doi.org/10.1109/CDC.2014.7039363
13. Sato, S., Waga, M.: tidySTL: Exposing implicit semantics in STL evaluation (extended version). arXiv preprint (2026)
14. Thomsen, A.K., Madsen, N.V.S., Evans, V.T., Wright, T.D., Esterle, L., Larsen, P.G.: mstlo: Efficient online monitoring of signal temporal logic. arXiv preprint arXiv:2605.26847 (2026)
15. Zhang, Z., An, J., Arcaini, P., Hasuo, I.: Caumon: A tool for online monitoring against signal temporal logic. Sci. Comput. Program. **253**, 103499 (2026). https://doi.org/10.1016/J.SCICO.2026.103499